PISCINE MAKERS · EMBEDDING LAB

Semantic Agent Engine

A memory that knows what was decided,
and what is still true.

Updated architecture · Jonathan Chayna · October 2026

The idea in one minute

In meetings, not everything is decided right away: people propose, hesitate, change their minds. A classic search finds sentences that **look alike**, but not necessarily the one that is **still true today**. This engine reads every exchange, files it in an organised memory, links pieces of information together ("this replaces that"), then finds the right version, with its proof.

It understands

A proposal is not a decision. "We could maybe…" triggers nothing; "it's approved" does.

It remembers

Every piece of information keeps its date, the person who said it and the exact original sentence.

It proposes, it never executes

Emails, tasks, events: always drafts to confirm. No address is ever invented.

The image to remember: a tidy workshop

You don't turn the whole workshop upside down to find a wrench: every tool is put in the right place **as soon as it arrives**. Here, every piece of information is filed next to the ones it relates to, the moment it arrives. Finding things becomes fast, and when something changes, the memory updates instead of piling up.

One core, two uses

Meetings and messages

The piscine's topic: find a decision, who committed, what changed between two meetings, prepare a file or an email draft.

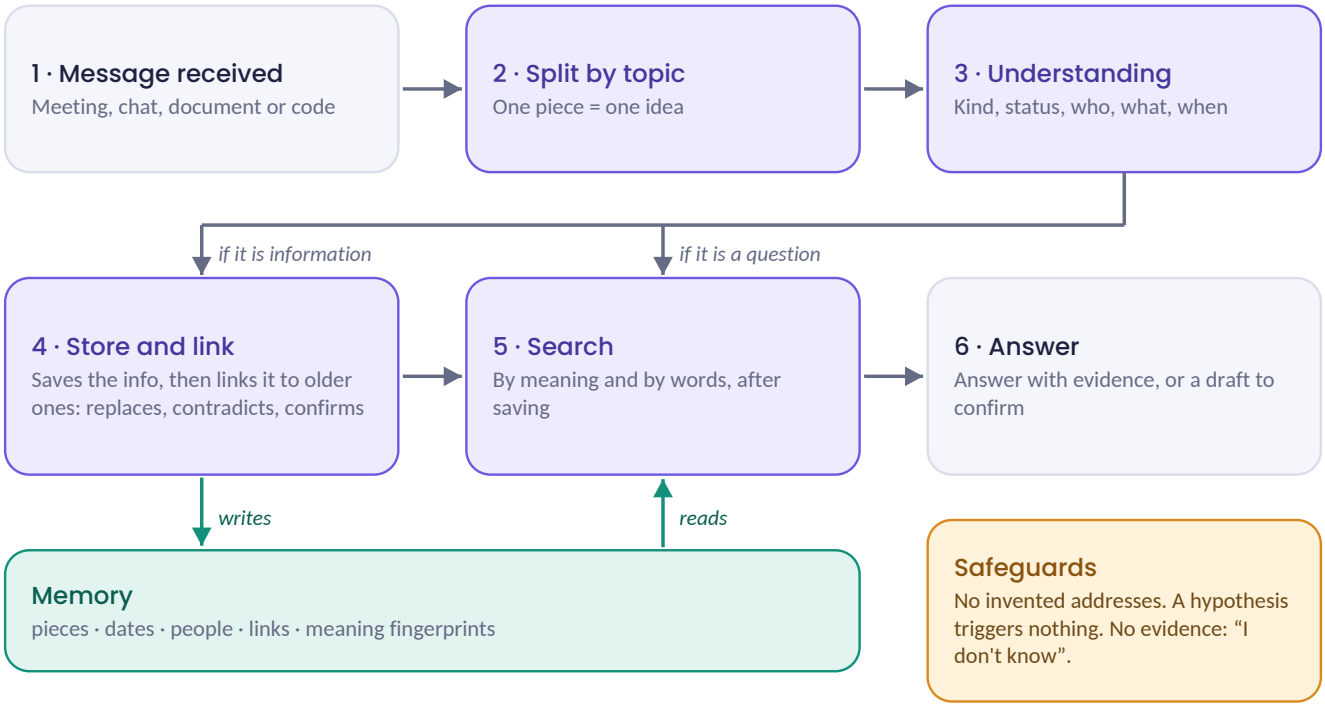
Documents and code

The same engine works as a general search: only the splitting changes (a coherent paragraph, a function). Memory, links and search stay identical.

1 The journey of a message

Here is what happens when a message arrives. Example used throughout this document:

> *"For project X, we're pushing the launch to December. By the way, where are we on the back end?"*



On the example

- **Piece A**: "we're pushing the launch to December". This is information (a decision). It is stored in memory and linked to the old launch date, which becomes "replaced".
- **Piece B**: "where are we on the back end?". This is a question: we search the memory **after** saving A, so the answer reflects what was just said.

2 Split by topic

One piece = the largest portion still about the same thing

Documents

Neighbouring sentences about the same thing are grouped. Five sentences on one subject: a single piece. One sentence about a car then one about a ball: two pieces.

Conversations

We cut as soon as the topic or intent changes, even mid-turn. In the example, a decision then a question: two pieces. A very short "OK, approved" stays attached to the previous message.

Code

One function = one piece. These are simple code-reading rules, no artificial intelligence.

Always: even when a piece groups several sentences, each original sentence is kept as **proof**. A big piece must never drown a detail such as a date or a name.

3 The memory

What we keep, and how it is filed for speed

For each piece, we keep

- the **text** and the **exact original sentence** (the proof)
- its **kind**: decision, action, commitment, question, answer, fact, risk, idea
- its **status**: proposed, confirmed, cancelled, replaced
- the **date**, the **person** speaking, the **owner** and the **deadline**
- the **source** (meeting, document) and the **project**
- **links** to other pieces: replaces, contradicts, confirms, answers
- a **meaning fingerprint**: a string of numbers that represents what the piece means

How it is filed

A notebook (SQLite database). Everything is written there: text, dates, people, statuses, links. Filtering by project or date is instant, and exact-word search is built in.

A meaning index (FAISS). Two texts with close meaning have close fingerprints; the index finds the closest in a few milliseconds. Each entry points to the piece's number in the notebook.

Golden rule: never erase. A replaced piece is marked "replaced". The history stays available to compare two meetings.

Why not plain JSON files? They are slow to filter and update as memory grows. JSON stays useful *inside* the notebook, for free-form extra information. Each piece has two fingerprints: a **stable** one (the meaning of the piece alone) and one **enriched by context**. **Verified:** the enriched fingerprint loses on real data → the stable one is the only one active.

4 Finding the right information

Six steps, from question to proven answer

1	**Understand the request** A small fast model guesses what the person wants: answer, compare, build a file, write an email, create a task, schedule, or ask for clarification. It also spots who, what, by when.
2	**Search two ways** By **meaning** (finds rephrasings) and by **exact words** (names, dates, numbers). The two lists are merged.
3	**Filter by request** Answer: only the information in force. Comparison: old and new, sorted by date. File: everything, with replaced information flagged.
4	**Break ties** A second model rereads the question with each candidate and ranks them from most to least relevant.
5	**Follow the replacement chain** If A was replaced by B, then by C, we keep C. The full history stays available.
6	**Answer with evidence** By default, a template writes the answer and cites every claim: **citable template answer by default; optional LLM rewrite, checked by a verifier** that bounces any rewrite containing an invented name or number. Not enough evidence: "I don't have enough information". For an email, the recipient is only kept if clear, recent and unique; otherwise we ask.

5 Updates

Fast right away, in depth later

Right away (instant)

- split and understand the new message
- save it in the memory
- compare it to its nearest neighbours and create links
- mark what is replaced as “replaced”

In the background (in batches, nightly)

- group pieces into discussion threads
- rebuild the meaning index
- **calibration audit**: guards against stale thresholds; it recalibrates and proposes, and never replaces the shipped model
- propose retraining once enough data has been corrected

This matters most for **messages**, which arrive continuously: the reaction is immediate, the big tidy-up happens in batches, without ever slowing the conversation.

6 What we train, and on what

Each model has a precise role and is measured separately

Role	Starting point	Trained on…	We measure…
Splitter where to cut	Sentence-similarity rule, then a small multilingual model	Pairs of neighbouring sentences “same topic / different topic”, built by gluing excerpts from different meetings, + annotated examples	Cut quality
Reader kind, status, who/what/when, request type	CamemBERT (moderncamembert-base), MIT licence verified	Annotated French conversations: public meetings + synthetic dialogues checked by hand	F1 per category; fall back to an LLM when “not sure”
Meaning engine fingerprints	Deterministic hash baseline (no download, no GPU); e5-base for real-French checks; bge-m3 was never measured	Question / right-passage pairs + “look-alikes” (same topic, wrong status, person or date)	Recall@K, MRR, nDCG; score on look-alikes. **Fine-tuned version trained:** wins its own tests, loses the final test → frozen base kept
Linker replaces, contradicts, confirms	Rules + deterministic guards (shipped). XNLI-trained head: wired in, but opt-in	Dated pairs: a decision before, a decision after, with the right relation	F1 per relation, confusion matrix
Final re-sorter ranking	bge-reranker-v2-m3, off the shelf	Nothing at first. Trained only if hard cases justify it	Gain on hard cases
“Concepts” layer tested, switched off	Small network that looks at all pieces together	Simulated streams: one item arrives, another replaces it	Measured on real French data: it loses, so it stays off. Proof that the judging rules work
Writer answers, files	Template that cites every claim (default); optional LLM rewrite (flag)	No training	Exact citations; verifier that rejects any invented name or number

Reader verified: Laya ruled out (a dialogue model, not an encoder); CamemBERT kept, MIT licence verified.

7 How this answers the brief

The brief's four difficulties, and the part that handles each

Brief's difficulty	What handles it
Relevance find it even with other words	Meaning engine + exact-word search + final re-sort.
Semantic precision proposal, decision, cancellation…	Status reader + “look-alikes” in training (same topic, wrong status).
Context several meetings, dates, versions	Linker (replaces / contradicts / confirms) + replacement chain + dated, sourced memory.
Intent and recipient which action, for whom	Request reader + fixed rule for the recipient (clear, recent, unique) + right to ask or to say “I don't know”.

8 How we know it works

- **Do we find the right information?** Recall@K and MRR, compared with a classic word search and an untrained meaning model.
- **Do we tell proposal from decision?** Score per category, with attention to negations, dates and numbers.
- **Do we link correctly?** Score per relation and confusion matrix (replaces / contradicts / confirms).
- **Do we pick the right recipient, or ask?** Precision, false-recipient rate, justified abstentions.
- **Are answers proven?** Exact citations and share of unsupported claims.

The decisive test: the version in force

We insert “launch in December” (October 1st), then “actually in January” (October 5th), then a few unrelated messages, and ask: “when is the launch?”. **Measured result:** the right version comes first in **0.55** of cases, versus **0.25** for a classic search, on unseen projects (+0.31, significant on 3 of 3 random seeds). The December → January test is in the report: geometry alone fails, the relation layer fixes it. **To be honest:** the absolute 0.90 target is not met, and on raw real speech topic tagging is still unsolved (more annotated meetings are needed). With hand-labelled topics, the 7 demos pass 10/10 on real French meetings.

9 Evidence

Shown, not promised

The demos pass

The 7 demos pass on the synthetic corpus, and 10/10 on three real French meetings.

Everything regenerates

Every number regenerates from the artefacts with a single command.

Licences verified

All models and all datasets carry a verified licence.

Measure	Result
Version in force first (unseen projects)	0.25 → 0.55 (+0.31; significant on 3 of 3 seeds) versus plain search
Absolute 0.90 target	Not met
7 demos	Passed on the synthetic corpus; 10/10 on three real French meetings (hand-labelled topics)
Fine-tuned meaning engine	Wins its own tests, loses the final test → frozen base kept
“Concepts” layer / enriched fingerprint	Loses on real French data → switched off

This is the brief's “reproducible protocol” requirement, **shown rather than promised**.

10 Plan and priorities

Week 1

- Data chosen and documented
- Annotation schema
- Word + meaning search, no training: the measured starting point

Week 2

- Train the reader and the linker
- Tune the meaning engine
- At least one ablation

Week 3

- Full agent, files, drafts
- 7-scene demo
- Report: results, errors, limits

If time runs short, the order is: (1) memory + hybrid search + reader + linker; (2) tuned meaning engine; (3) learned splitter; (4) “concepts” layer as a compared experiment. The core works without (4).

11 Measured limits

What the measurements show, not what we feared

- **“Contradicts” is the weakest relation.** The new confusion matrix shows its shape: that is where the linker errs most.
- **3 of 12 unanswerable questions still leak an answer** instead of an “I don't know”.
- **The “answers” relation is never produced** by the shipped linker.
- **Topic tagging on raw real speech** is the one open problem tied to data scale: more annotated meetings are needed.

12 Short glossary

Technical words, in plain language

Meaning fingerprint (embedding)	A string of numbers that represents the meaning of a text. Two texts with close meaning have close fingerprints.
Word search (BM25)	Finds texts containing the same words as the question. Very good for names, dates and numbers.
Fusion (RRF)	Merges two rankings (by meaning and by words) into one, without having to tune weights.
Re-sorter (reranker)	A model that reads the question and a candidate together to rank them better.
Linker	Compares two pieces and says whether they confirm each other, contradict, or one replaces the other.
Look-alikes (hard negatives)	Examples that look alike but do not mean the same thing: same topic, wrong status, wrong date.
Train (fine-tuning)	Take an already-trained model and specialise it on our own examples.
FAISS / SQLite	Two filing tools: FAISS finds by closeness of meaning, SQLite stores everything else in a notebook.
LLM	A language model that writes text. Here it is optional, and a verifier checks what it writes.